Exercícios Desempenho

1.

```
R:
     addi x5, x7,-5
     add x5, x5, x6
```

2. R:

```
     f = g+h+i
```

3. R:

```
     sub x30, x28, x29
     slli x30, x30, 3
     add x3, x30, x10
     ld x30, 0(x3)
     sd x30, 64(x11)
```

4. R:

```
     B[g]= A[f] + A[f+1]
```

5. R:

Little-Endian		Big-Endian	
Address	Data	Address	Data
3	ab	3	12
2	cd	2	ef
1	ef	1	cd
0	12	0	ab

6. R:

   a. R: 0x50000000
   b. R: houve estouro
   c. R: 0xB0000000
   d. R: não houve estouro
   e. R: 0xD0000000
   f. R: houve estouro

7.

   a. R: 0x23456780
   b. R: 0x123efef8
   c. R: 0x45

8. R:

```
ld x6, 0(x17)
slli x6, x6, 4
```

9.

a. R: [0x1ff00000, 0x200FFFFE]

b. R: [0x1FFFF000, 0x20000ffe]

10.

a. R: formato do JAL

b. R:

```
loop:
     addi x29, x29, -1
     bgt x29, x0, loop
     addi  x29, x29, 1
```

11. R:

```
LOOPI:
          addi x7, x0, 0
          bge  x7, x5, ENDI
          addi x30, x10, 0
          addi x29, x0, 0
LOOPJ:
          bge  x29, x6, ENDJ
          add x31, x7, x29
          sw x31, 0(x30)
          addi x30, x30, 16
          addi x29, x29, 1
          jal x0, LOOPJ
ENDJ:
          addi x7, x7, 1
          jal x0, LOOPI
ENDI:
```

12. R:

```
int i;
     for (i = 0; i < 100; i++) {
          result += *MemArray;
          MemArray++;
     }
     return result;

// outra maneira:

int i;
for (i = 0; i < 100; i++)
     result += MemArray[i];
return result;
```

13. R: O endereço do último elemento do MemArray pode ser usado para encerrar o loop:

```
     add x29, x10, 800 // x29 = &MemArray[101]
LOOP:
     ld x7, 0(x10)
     add x5, x5, x7
     addi x10, x10, 8
     blt x10, x29, LOOP
```

14. R: o código abaixo supõe palavras de 64 bits.

```
fib:
      beq    x10,   x0, done  // If n==0, return 0
      addi   x5,    x0, 1
      beq    x10,   x5, done  // If n==1, return 1
      addi   x2,    x2, -16   // Allocate 2 words of stack
                                 space
      sd     x1,    0(x2)     // Save the return address
      sd     x10,   8(x2)     // Save the current n
      addi   x10,   x10, -1   // x10 = n-1
      jal    x1,    fib       // fib(n-1)
      ld     x5,    8(x2)     // Load old n from the stack
      sd     x10,   8(x2)     // Push fib(n-1) onto the stack
      addi   x10,   x5, -2    // x10 = n-2
      jal    x1,    fib       // Call fib(n-2)
      ld     x5,    8(x2)     // x5 = fib(n-1)
      add    x10,   x10, x5   // x10 = fib(n-1)+fib(n-2)
// Clean up:
      ld     x1,    0(x2)     // Load saved return address
      addi   x2,    x2, 16    // Pop two words from the stack
done:
      jalr   x0,    x1
```